GEO5017 - Assignment 1: Linear Regression

Lars van Dijk (5571308), Niranjan Pradeep (6401643) and Thys Kok (5591406)

March 5, 2026

1 Basic model

Solution 1: Before working with the data points, we must verify that x_i and t_i show a linear relationship and also identify any outliers; creating a scatter plot of the data will give us a clearer idea of the dataset.

Solution 2: The optimal solution is the set of parameters or weights that minimizes the sum of squared error between the observed data points and the regression line.

2 Programming: drone

2.1

Given the set of data points, we made use of the 'matplotlib' python library to plot the path followed by the drone in the 3-D space (Figure 1).

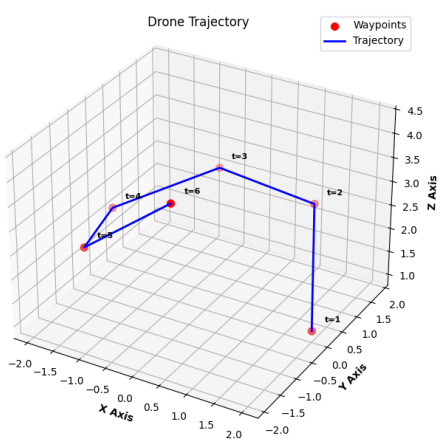

Figure 1: Observed drone positions over the course of $T = \{1, 2, 3, 4, 5, 6\}$.

2.2

a. Assuming constant velocity, the drone position can be modelled per spatial dimension using a linear polynomial:

$$p(t) = vt + b$$

where v represents the velocity and b the starting position which both need to be determined. Furthermore, the time t and the corresponding position p are given. Per axis this results in:

$$x(t) = v_x t + b_x$$
$$y(t) = v_y t + b_y$$
$$z(t) = v_z t + b_z$$

We assume that the velocity is constant per dimension. Using gradient descent we can determine the values of v and b such that the model represents the given datapoints as closely as possible. How well the model represents the data is measured using the SSE formula:

$$SSE = \sum_{i=1}^{N} (p_i - (vt_i + b))^2$$

The learning rate and maximum number of iterations were determined empirically by testing values of multiple sizes and taking the ones that give the smallest SSE. The stopping criterion is either the maximum number of iterations being reached or the improvement falling below a certain tolerance. The fastest convergence was achieved with a learning rate of 0.01 and a maximum number of iterations of 500. A value of 0.001 for the tolerance has the gradient descent already converge after 159, 177 and 134 iterations respectively for the x-, y- and z-axis. For smaller tolerances the improvement in the SSE is negligible, and for larger tolerances, the error increases substantially. Hence, the value of 0.001 was used. In the end, the determined values for the velocity and starting position per dimension are as following:

$$v_x = -0.429,\ b_x = 1.418$$
$$v_y = -0.579,\ b_y = 2.078$$
$$v_z = 0.494,\ b_z = 0.753$$

These values result in a SSE of 15.99 of the estimated positions using the above values for v and b. See below for a plot of the constant velocity model in figure 2.

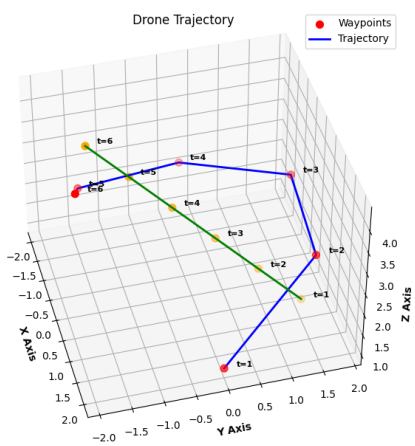

Figure 2: Observed drone positions and predicted positions using the constant velocity regression model.

b. Assuming constant acceleration, the drone position in each spatial dimension is modelled using a quadratic polynomial:

$$p(t) = at^2 + vt + b$$

where a represents acceleration, v the velocity term, and b the initial position. For the three axes this results in:

$$x(t) = a_x t^2 + v_x t + b_x$$
$$y(t) = a_y t^2 + v_y t + b_y$$
$$z(t) = a_z t^2 + v_z t + b_z$$

Gradient descent was used to estimate the model parameters by minimising the residual error between the observed and predicted positions. The learning rate used in this function is selected by evaluating multiple values and selecting the configuration that produced the lowest residual error. The gradient descent solver uses two stopping conditions: a maximum number of iterations is reached, or the parameter updates are smaller than the tolerance. The tolerance was set to

10^{-7}. A smaller tolerance would only lead to additional iterations while producing negligible changes in the residual error, indicating that the solution had already effectively converged. For the constant acceleration model, the best performance was obtained with a learning rate of 0.0001. The best configuration allowed a maximum of 200,000 iterations, while the solver converged for all axes after 175,956 iterations. The following model parameters are obtained, resulting in the following equations.

$$
\begin{aligned}
x(t) &= 0.4334\,t^2 - 3.4753\,t + 5.5138 \\
y(t) &= -0.3234\,t^2 + 1.6733\,t - 0.8918 \\
z(t) &= 0.0546\,t^2 + 0.1002\,t + 1.3088 \\
SSE &= \sum_{i=1}^{N} (p_i - (at_i^2 + vt_i + b))^2
\end{aligned}
$$

The residual error calculated using the SSE formula is 4.942. Comparing this result to the residual error of the constant velocity model, it shows that by introducing the additional parameter a, the residual error is decreased. This shows that the constant acceleration model better fits the trajectory data.

The model could potentially be improved by increasing the model complexity by fitting a higher-order polynomial. However, since the dataset contains only a small number of observations, increasing the model complexity could lead to overfitting.

c. Using the regression model defined in the last question. The most likely position at $t = 7$ can be calculated by filling in the formulas:

$$
\begin{aligned}
x(7) &= 0.4334(7)^2 - 3.4753(7) + 5.5138 = 2.423 \\
y(7) &= -0.3234(7)^2 + 1.6733(7) - 0.8918 = -5.025 \\
z(7) &= 0.0546(7)^2 + 0.1002(7) + 1.3088 = 4.688
\end{aligned}
$$

This gives the predicted drone position at $t = 7$

$$P(7) = (2.423,\ -5.025,\ 4.688)$$

The observed drone positions and the predicted positions are plotted to visualise the estimated trajectory of the drone (see Figure 3).

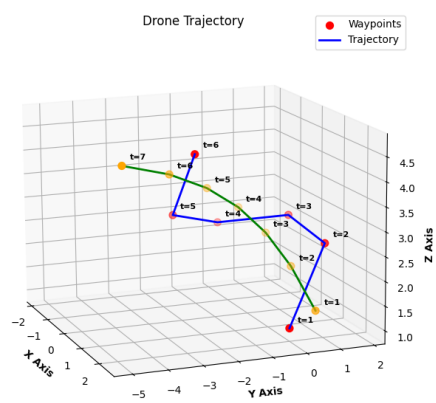

Figure 3: Observed drone positions and predicted positions using the constant acceleration regression model.

3 Code

Link to GitHub project repository.

4 Division of Work

Table 1: Division of the Tasks

Lars van Dijk (30%)	**Niranjan Pradeep (30%)**	**Thys Kok (40%)**
Questions 2.2b and 2.2c Constant acceleration gradient descent	Question 1.1, 1.2, 2.1 Plotter functions Added closed-form regression checker (Linear).	Question 2.2a Constant velocity gradient descent General repository organization